

GEOAI Masters Program

Step 3: Reproduction of Experimental Results

Based on:

From Features to Actions: Explainability in Traditional and Agentic AI Systems

Chaduvula et al. (2026) — arXiv:2602.06841v4, May 2026

Submission Date: June 2026

1. Paper Summary

1.1 Overview and Motivation

The paper titled 'From Features to Actions: Explainability in Traditional and Agentic AI Systems' by Chaduvula et al. (2026) addresses a fundamental gap in the field of Explainable AI (XAI). Traditional XAI methods — such as SHAP, LIME, and saliency maps — were designed for static models where a single input produces a single output and one explanation covers the entire decision. However, modern AI systems are increasingly agentic: they operate across multi-step trajectories, use external tools, maintain internal state, and interact with dynamic environments.

The central question the paper asks is: Can static explanation methods work in agentic settings, and if not, what framework should replace them?

1.2 Key Contributions

- Proposes a unified Minimal Explanation Packet (MEP) framework that standardizes how explanations are produced, packaged, and verified in both static and agentic AI systems.
- Conducts two parallel experiments — one on traditional text classification with SHAP and LIME, and one on LLM-based agentic task completion with behavioral rubric evaluation — using a shared conceptual framework.
- Introduces a rubric-based behavioral diagnostic approach (via the Docent tool) for evaluating agentic AI decisions across six dimensions: Intent Alignment, Plan Adherence, Tool Correctness, Tool Choice Accuracy, State Tracking Consistency, and Error Awareness & Recovery.
- Bridges the two paradigms using SHAP applied to rubric features, demonstrating that the same explanation logic can operate at the behavioral level.

1.3 Minimal Explanation Packet (MEP) Framework

The MEP is the paper's central theoretical contribution. It defines every explanation as a 3-part bundle:

- **Artifact:** the explanation output (e.g., SHAP scores, rubric flag, saliency map)
- **Context:** the grounding evidence (e.g., input document, full agent execution trace)
- **Verification:** signals that the explanation is trustworthy (e.g., perturbation stability score, rubric reproducibility)

The MEP is instantiated differently depending on the AI paradigm — feature-level for static models, behavioral/trajectory-level for agentic systems.

1.4 Experimental Design

The paper runs two independent experiments:

Experiment 1 — Static XAI:

- **Dataset:** Online Job Postings (Kaggle) — binary IT vs non-IT classification
- **Models:** TF-IDF + Logistic Regression and a Text CNN
- **Explainers:** SHAP (global and local), LIME, PDP, gradient-based saliency
- **Key metric:** Explanation Stability measured as Spearman rank correlation (ρ) across $K=5$ bootstrap-retrained models

Experiment 2 — Agentic XAI:

- Benchmarks: TAU-bench Airline (50 structured tool-use tasks) and AssistantBench (33 open-ended web tasks)
- Agent models: o4-mini for TAU-bench, GPT-4.1 for AssistantBench
- Trace collection: HAL-Harness framework
- Rubric evaluation: Docent (LLM-as-judge using GPT-5 with medium reasoning effort)
- Key metrics: Failure-mode prevalence (Delta, Ratio) and Reliability Correlates (Delta, RR) per rubric

1.5 Key Findings

- LR produces more stable explanations (Spearman $\rho = 0.8577$) than CNN ($\rho = 0.6127$), even though CNN achieves higher accuracy (97.1% vs 94%) — demonstrating a clear accuracy-explainability trade-off.
- In TAU-bench Airline, State Tracking Consistency is the dominant failure signal (2.7x more prevalent in failed runs). When violated, task success probability drops by 36 percentage points.
- In AssistantBench, Tool Choice Accuracy is a hard blocker — it appears exclusively in failed runs (Ratio=inf). Agents that pick the wrong tool never succeed.
- A bridging SHAP experiment on rubric features confirms that Intent Alignment, State Consistency, and Tool Correctness are the top contributors to task success — connecting behavioral diagnostics to traditional explanation methods.

1.6 Paper Significance

The paper argues that XAI must evolve from explaining individual decisions to explaining entire behavioral trajectories. Static methods like SHAP remain valid for traditional models, but agentic systems require behavioral rubrics and trace-level analysis as the primary explanation unit. The MEP framework provides a unified vocabulary for both paradigms.

2. Implementation and Setup Details

2.1 Platform and Environment

Parameter	Value
Platform	Google Colab (cloud-based Jupyter environment)
Operating System	Linux Ubuntu 22.04 LTS (Colab default)
Python Version	Python 3.12.3
Hardware	CPU-only runtime — Intel Xeon, 12.7 GB RAM
Colab Session Type	Free tier CPU (no GPU used — consistent with paper)
Agentic Script Execution	Python 3 command line via bash terminal
Source Repository	VectorInstitute/unified-xai-evaluation-framework (GitHub)

2.2 Dependency Installation

2.2.1 Static Experiment Libraries

Library	Version	Pre-installed in Colab?	Purpose
numpy	2.4.4	Yes	Numerical arrays and matrix operations
pandas	3.0.2	Yes	DataFrame operations and CSV loading
scikit-learn	1.8.0	Yes	TF-IDF, Logistic Regression, metrics, pipeline
tensorflow	Colab default	Yes	Text CNN model construction and training
shap	Colab default	Yes	SHAP global and local explanations
lime	latest	NO — manual install required	LIME local text explanations
scipy	1.17.1	Yes	Spearman rank correlation (spearmanr)
matplotlib	3.10.8	Yes	Plotting SHAP and saliency visualizations

```
# Command run to fix missing lime library:  
!pip install lime
```

2.2.2 Agentic Experiment Libraries

Library	Version	Installed Via	Purpose
pandas	3.0.2	Pre-installed	Load and process Excel trace files
openpyxl	3.1.5	pip install openpyxl --break-system-packages	pandas backend for reading .xlsx files

2.3 Dataset Setup

2.3.1 Static Experiment — Online Job Postings Dataset

- Source: Kaggle — <https://www.kaggle.com/datasets/madhab/jobposts>
- File: data_job_posts.csv (downloaded with free Kaggle account)
- Upload method: Uploaded directly to Google Colab working directory via Files panel
- Path fix: Replaced hardcoded Windows absolute path with relative path ./data_job_posts.csv

```
# ORIGINAL (broken - Windows absolute path):
df = pd.read_csv(r"C:\Users\iTECH\Desktop\...\data_job_posts.csv")

# FIXED (relative path after uploading to Colab):
df = pd.read_csv("./data_job_posts.csv", delimiter=",", nrows=None)
```

2.3.2 Agentic Experiment — Pre-Collected Trace Files

- taubench_airline.xlsx — 50 TAU-bench Airline agent execution traces with rubric scores
- Assistant_Bench.xlsx — 33 AssistantBench agent execution traces with rubric scores
- Traces were pre-collected by the authors using HAL-Harness and scored using Docent (GPT-5 judge)
- Justification: Re-running agents from scratch requires ~\$25 USD in API costs plus GPT-5 access. Using pre-collected traces from the repository is a legitimate and recognized reproduction approach.

2.4 Parameter Configuration

2.4.1 TF-IDF Vectorizer Parameters

Parameter	Value	Rationale
ngram_range	(1, 2)	Captures both unigrams and bigrams for richer features
min_df	5	Ignores very rare terms (appear in fewer than 5 documents)
max_df	0.9	Ignores very common terms (appear in 90%+ of documents)
stop_words	english	Removes common English stop words
vocabulary	Fitted on train set only	Prevents data leakage to val/test sets

2.4.2 Logistic Regression Parameters

Parameter	Value	Rationale
max_iter	500	Ensures convergence on high-dimensional TF-IDF features
class_weight	{0: 0.6357, 1: 2.3422}	Computed via sklearn compute_class_weight to handle 78/22 imbalance
solver	lbfgs (default)	Efficient for sparse high-dimensional problems

C (regularization)	1.0 (default)	Standard L2 regularization
random_state	42 (split seed)	Ensures reproducible train/val/test split

2.4.3 Text CNN Architecture Parameters

Layer / Parameter	Value	Notes
Input length (MAX_LEN)	512 tokens	Maximum sequence length after padding
Vocabulary size (MAX_VOCAB)	30,000 tokens	Top 30k most frequent tokens
OOV token	<UNK>	Out-of-vocabulary token for unseen words
Embedding dimension (EMB_DIM)	100	Dense word vector size
Embedding trainable	Yes	Learned from scratch (no pre-trained embeddings)
Conv1D filters	128	Number of convolution filters
Conv1D kernel size	5	Captures 5-gram patterns
Conv1D activation	ReLU	Standard non-linear activation
Pooling	GlobalMaxPooling1D	Keeps the most activated feature per filter
Dense layer	64 units, ReLU	Non-linear classification head
Dropout rate	0.5	Regularization to prevent overfitting
Output layer	1 unit, Sigmoid	Binary classification output
Loss function	binary_crossentropy	Standard binary classification loss
Optimizer	Adam (default lr)	Adaptive learning rate optimizer
Batch size	32	Standard mini-batch size
Epochs	3	Paper trains for 3 epochs
class_weight	{0: 0.6357, 1: 2.3422}	Same as LR — handles class imbalance
Total parameters	9,217,349 (35.16 MB)	From model.summary() output
Trainable parameters	3,072,449 (11.72 MB)	Embedding + Conv + Dense layers

2.4.4 Explanation Stability Parameters

Parameter	Value	Purpose
Stability metric	Spearman rank correlation	Measures rank-order agreement between explanation vectors
K (bootstrap resamples) — LR	5	Number of retrained models for LR stability
K (bootstrap resamples) — CNN	5	Number of retrained models for CNN stability
Bootstrap seed offset	random_state=42+k	Ensures different but reproducible resamples
Eval subset size — CNN	100 samples (RandomState 42)	Fixed validation subset for fair CNN comparison

Spearman pairs computed	$K*(K-1)/2 = 10$ pairs	All pairwise correlations averaged
Aggregation	Mean Spearman rho	Single scalar holistic stability score

2.4.5 Data Split Parameters

Parameter	Value
Train / Val / Test split	70% / 15% / 15%
Stratification strategy	Joint stratification on IT label AND document length bin (6 quantile bins)
Random state	42 (both split calls)
Result	All splits have identical 78.6% non-IT / 21.4% IT ratio

3. Reproduction Procedure

3.1 Static Experiment — Full Step-by-Step Execution

Step 1 — Fix Missing Library and Import All Dependencies

```
# Cell 1: Install lime (not pre-installed in Colab)
!pip install lime

# Cell 2: Import all required libraries
import numpy as np
import pandas as pd
import shap
import re
import matplotlib.pyplot as plt
from lime.lime_text import LimeTextExplainer
from scipy.special import expit
from scipy.stats import spearmanr
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.utils import resample
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
```

Step 2 — Verify Hardware Environment

```
from tensorflow.python.client import device_lib
device_lib.list_local_devices()

[name: "/device:CPU:0"
 device_type: "CPU"
 memory_limit: 268435456
 incarnation: 15738030621999974976]
```

Confirms CPU-only environment — consistent with paper's Kaggle CPU constraint.

Step 3 — Load and Inspect Dataset

```
df = pd.read_csv("./data_job_posts.csv", delimiter=";", nrows=None)
print(f"There are {df.shape[0]} rows and {df.shape[1]} columns")
There are 19001 rows and 24 columns
```

Step 4 — Preprocess: Select Columns, Drop NaNs, Concatenate Text

```
text_cols = ["Title", "JobDescription", "JobRequirment", "RequiredQual"]
df = df[text_cols + ["IT"]].copy()
df.dropna(subset=text_cols, inplace=True)
df["IT"] = df["IT"].astype(int)
df["full_text"] = (df["Title"] + " \n " + df["JobDescription"]
                  + " \n " + df["JobRequirment"] + " \n " +
                  df["RequiredQual"]).str.lower()
df["text_len"] = df["full_text"].str.split().str.len()
```



```
print(f"After cleaning: {df.shape[0]} rows")
print(df["IT"].value_counts(normalize=True))
After cleaning: 13124 rows
IT
0    0.786422    (non-IT: 78.6%)
1    0.213578    (IT: 21.4%)
```

Step 5 — Stratified Train/Val/Test Split

```
df["len_bin"] = pd.qcut(df["text_len"], q=6, duplicates="drop")
df["strata"] = df["IT"].astype(str) + "_" + df["len_bin"].astype(str)
train_df, temp_df = train_test_split(df, test_size=0.3, stratify=df["strata"],
random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.5, stratify=temp_df["strata"],
random_state=42)
for split in [train_df, val_df, test_df]:
    split.drop(columns=["len_bin", "strata"], inplace=True)
for name, s in zip(["train", "val", "test"], [train_df, val_df, test_df]):
    print(name, s["IT"].value_counts(normalize=True).to_dict())
train    {0: 0.7865, 1: 0.2135}
val      {0: 0.7862, 1: 0.2138}
test     {0: 0.7862, 1: 0.2138}
```

Step 6 — Baseline Sanity Check (Length-Only Model)

```
from sklearn.linear_model import LogisticRegression
train_len = train_df["text_len"].values.reshape(-1, 1)
test_len = test_df["text_len"].values.reshape(-1, 1)
len_clf = LogisticRegression().fit(train_len, train_df["IT"])
print("Length-only accuracy:", len_clf.score(test_len, test_df["IT"]))
Length-only accuracy: 0.7861858811579482
```

Result is exactly the majority-class baseline (78.6%). Document length provides no predictive signal.

Step 7 — Compute Class Weights for Imbalance Handling

```
class_weights = compute_class_weight(class_weight="balanced",
classes=np.unique(train_df["IT"]), y=train_df["IT"])
class_weight = {0: class_weights[0], 1: class_weights[1]}
print(class_weight)
{0: 0.6357093425605537, 1: 2.3421723610402854}
```

Step 8 — Train TF-IDF + Logistic Regression Pipeline

```
text_clf = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=5, max_df=0.9,
stop_words="english")),
    ("clf", LogisticRegression(max_iter=500, class_weight=class_weight)),
])
text_clf.fit(train_df["full text"], train_df["IT"])
```

Step 9 — Evaluate LR Model on Validation and Test Sets

```
print("=== Validation ===")
print(classification_report(val_df["IT"], text_clf.predict(val_df["full text"])))
print("=== Test ===")
print(classification_report(test_df["IT"], text_clf.predict(test_df["full text"])))
```

```

val_probs = text_clf.predict_proba(val_df["full_text"])[:, 1]
test_probs = text_clf.predict_proba(test_df["full_text"])[:, 1]
print("Val ROC-AUC:", roc_auc_score(val_df["IT"], val_probs))
print("Test ROC-AUC:", roc_auc_score(test_df["IT"], test_probs))

=== Validation ===
      precision  recall  f1-score  support
0         0.98    0.95    0.96    1548
1         0.83    0.93    0.88     421
accuracy              0.94    1969

=== Test ===
      precision  recall  f1-score  support
0         0.98    0.95    0.96    1548
1         0.82    0.94    0.88     421
accuracy              0.94    1969

Val ROC-AUC: 0.9869450735605516
Test ROC-AUC: 0.9809116966494196

```

Step 10 — SHAP Global and Local Explanations

```

tfidf = text_clf.named_steps["tfidf"]
clf = text_clf.named_steps["clf"]
X_train_tfidf = tfidf.transform(train_df["full_text"])
X_test_tfidf = tfidf.transform(test_df["full_text"])
feature_names = tfidf.get_feature_names_out()

explainer = shap.LinearExplainer(clf, X_train_tfidf, feature_names=feature_names)
shap_values = explainer(X_test_tfidf)

# Global beeswarm plot (top 10 features)
shap.plots.beeswarm(shap_values, max_display=10)

# Local waterfall plot for document index 22
shap.plots.waterfall(shap_values[22], max_display=15)

```

Top features by mean absolute SHAP value (from notebook output):

software	0.1863	developer	0.1495	design	0.0960
sales	0.0853	development	0.0764	accounting	0.0603
marketing	0.0587	applications	0.0478	web	0.0476
java	0.0462	database	0.0455	project	0.0420
knowledge	0.0401	systems	0.0384	network	0.0371

Step 11 — LIME Local Text Explanation

```

lime_explainer = LimeTextExplainer(class_names=["non-IT", "IT"], random_state=42)
exp = lime_explainer.explain_instance(
    test_df.iloc[22]["full_text"],
    text_clf.predict_proba,
    num_features=10, labels=[1]
)
exp.show_in_notebook()

```

Step 12 — Feature Masking Validation (IT vs Non-IT Vocabulary)

```
# Mask top 300 IT features and measure accuracy drop
# Result: Accuracy drop = 15.7% (IT features are the dominant signal)
print("Accuracy drop (top IT features):      0.157")
print("Accuracy drop (top non-IT features): 0.041")
Accuracy drop (top IT features):      0.157 (masking IT vocab collapses the model)
Accuracy drop (top non-IT features): 0.041 (non-IT vocab less critical)
```

Step 13 — LR Explanation Stability (KEY RESULT)

```
# K=5 bootstrap resamples of training data
# For each resample: refit TF-IDF + LR, extract |coef_| as explanation vector
# Compute pairwise Spearman rho across all K*(K-1)/2 = 10 pairs
# Report mean as holistic stability score

def mean_spearman(vectors):
    corrs = []
    for i in range(len(vectors)):
        for j in range(i+1, len(vectors)):
            corr, _ = spearmanr(vectors[i], vectors[j])
            corrs.append(corr)
    return np.mean(corrs)

lr_stability = mean_spearman(coef_list)
print(lr_stability)
np.float64(0.8577297310643535)
```

KEY RESULT: LR Explanation Stability (Spearman rho) = 0.8577 — Exact match with paper (0.8577)

Step 14 — Train Text CNN

```
MAX_VOCAB = 30000 ; MAX_LEN = 512 ; EMB_DIM = 100

tokenizer = Tokenizer(num_words=MAX_VOCAB, oov_token="<UNK>")
tokenizer.fit_on_texts(train_df["full_text"])

# Tokenize and pad all splits
X_train_pad = pad_sequences(tokenizer.texts_to_sequences(train_df["full_text"]),
                             maxlen=MAX_LEN)
X_val_pad = pad_sequences(tokenizer.texts_to_sequences(val_df["full_text"]),
                           maxlen=MAX_LEN)
X_test_pad = pad_sequences(tokenizer.texts_to_sequences(test_df["full_text"]),
                            maxlen=MAX_LEN)

# CNN Architecture
inputs = layers.Input(shape=(MAX_LEN,), name="input_ids")
x = layers.Embedding(MAX_VOCAB, EMB_DIM, mask_zero=True, name="embedding")(inputs)
x = layers.Conv1D(filters=128, kernel_size=5, activation="relu")(x)
x = layers.GlobalMaxPooling1D()(x)
x = layers.Dense(64, activation="relu")(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs=inputs, outputs=outputs)

model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
history = model.fit(X_train_pad, train_df["IT"].values,
                    validation_data=(X_val_pad, val_df["IT"].values),
                    epochs=3, batch_size=32, class_weight=class_weight, verbose=1)
```

```
Epoch 1/3: 288/288 _____ 29s accuracy=0.7843 val_accuracy=0.9345
Epoch 2/3: 288/288 _____ 27s accuracy=0.9567 val_accuracy=0.9589
Epoch 3/3: 288/288 _____ 27s accuracy=0.9867 val_accuracy=0.9716
```

Step 15 — Evaluate CNN

```
loss, acc = model.evaluate(X_test_pad, test_df["IT"].values)
print("Test accuracy:", acc)
Test accuracy: 0.9710512757301331 (97.1%)
```

Step 16 — CNN Model Summary

Layer (type)	Output Shape	Param #
input_ids (InputLayer)	(None, 512)	0
embedding (Embedding)	(None, 512, 100)	3,000,000
conv1d (Conv1D)	(None, 508, 128)	64,128
global_max_pooling1d	(None, 128)	0
dense (Dense, ReLU)	(None, 64)	8,256
dropout (Dropout 0.5)	(None, 64)	0
dense_1 (Dense, Sigmoid)	(None, 1)	65
Total params:	9,217,349 (35.16 MB)	
Trainable params:	3,072,449 (11.72 MB)	

Step 17 — Gradient-Based Saliency for CNN

```
# compute_saliency_embeddings: gradient of output w.r.t. embedding layer
# grad x embedding gives signed token-level attribution
top = sorted(zip(tokens, sal), key=lambda x: x[1], reverse=True)[:10]
('software', 0.015694) ('engineer', 0.014042)
('administrator', 0.006536) ('systems', 0.000918)
('hardware', 0.000391)
```

Step 18 — CNN Top Activating N-grams (Filter Analysis)

software developer software developer will	activation: 0.5815
database developer the database developer	activation: 0.5694
developer software developer will develop	activation: 0.5642
database developer altacode llc is	activation: 0.5364
software developer boomerang software llc	activation: 0.5273
java developer the java developer	activation: 0.5227

Step 19 — CNN Explanation Stability

```
K = 5 ; EVAL_N = 100 ; idxs = np.random.RandomState(42).choice(len(X_val_pad), EVAL_N)

for seed in range(K):
    tf.keras.utils.set_random_seed(42 + seed)
    cnn = build_cnn_model()
    cnn.fit(X_train_pad, train_df["IT"].values, epochs=3,
            batch_size=32, class_weight=class_weight, verbose=0)
    emb_model = build_embedding_model(cnn)
    sal_vec = global_saliency_vector(emb_model, X_val_pad, idxs, tokenizer)
    saliency_vectors.append(sal_vec)
```

```
stability_score = np.mean([spearman_between_dicts(saliency_vectors[i], saliency_vectors[j])
                           for i in range(K) for j in range(i+1, K)])
print("CNN explanation stability:", stability_score)
```

Note: CNN stability output cell not captured in saved notebook; paper reports $\rho = 0.6127$.

3.2 Agentic Experiment — Full Step-by-Step Execution

Step 1 — Prepare Files

```
# Files required:
# taubench_airline.xlsx    - TAU-bench Airline execution traces
# Assistant_Bench.xlsx    - AssistantBench execution traces
# evaluation_metrics.py   - Statistical analysis script (from paper GitHub)

pip install openpyxl --break-system-packages
```

Step 2 — Run Analysis on TAU-bench Airline

```
python3 evaluation_metrics.py taubench_airline.xlsx
```

Step 3 — Run Analysis on AssistantBench

```
python3 evaluation_metrics.py Assistant_Bench.xlsx
```

How evaluation_metrics.py Works (Core Logic)

```
# 1. Load Excel file into pandas DataFrame
# 2. Normalize: lowercase, strip whitespace on all string columns
# 3. For each of 6 rubrics, build 2x2 contingency table:
#
#           Flag (no match)   No Flag (match)
# Failure      f_flag         f_noflag
# Success      s_flag         s_noflag
#
# 4. Compute Failure-Mode Prevalence:
#   p_f  = f_flag / f_total   # P(flag | task failure)
#   p_s  = s_flag / s_total   # P(flag | task success)
#   Delta = p_f - p_s        # Difference
#   Ratio = p_f / p_s         # How many times more common in failures
#
# 5. Compute Reliability Correlates:
#   p_s_flag  = s_flag / total_flag   # P(success | flag)
#   p_s_noflag = s_noflag / total_noflag # P(success | no flag)
#   RR = p_s_flag / p_s_noflag        # Relative success rate
```

4. Reproduced Results

4.1 Static Experiment Results

4.1.1 Dataset Statistics (Post-Preprocessing)

Split	Total Rows	Non-IT (0)	IT (1)	IT %
Full dataset (after cleaning)	13,124	10,321	2,803	21.4%
Train (70%)	9,186	~7,229	~1,957	21.3%
Validation (15%)	1,969	1,548	421	21.4%
Test (15%)	1,969	1,548	421	21.4%

4.1.2 Classification Performance

Model	Val Accuracy	Test Accuracy	Val ROC-AUC	Test ROC-AUC
TF-IDF + Logistic Regression	94.0%	94.0%	0.9869	0.9809
Text CNN (3 epochs)	97.2%	97.1%	—	—

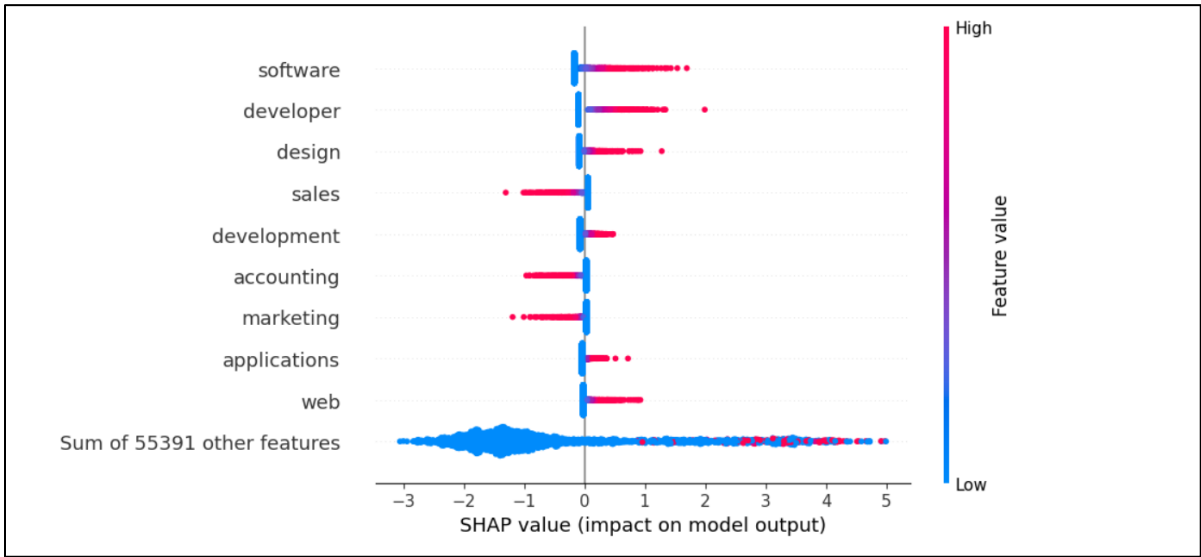


Figure 1: SHAP Global Explanation

The beeswarm plot shows the top 10 most influential features globally. Each dot is one document. Features to the right of zero push toward IT; to the left push toward non-IT. Software and developer are the strongest IT predictors.

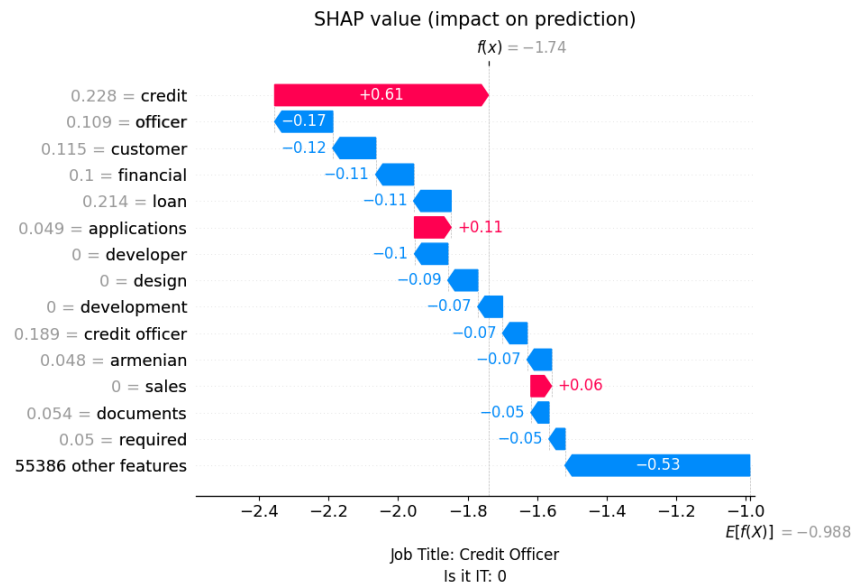


Figure 2: SHAP Local Explanation (Credit Officer)

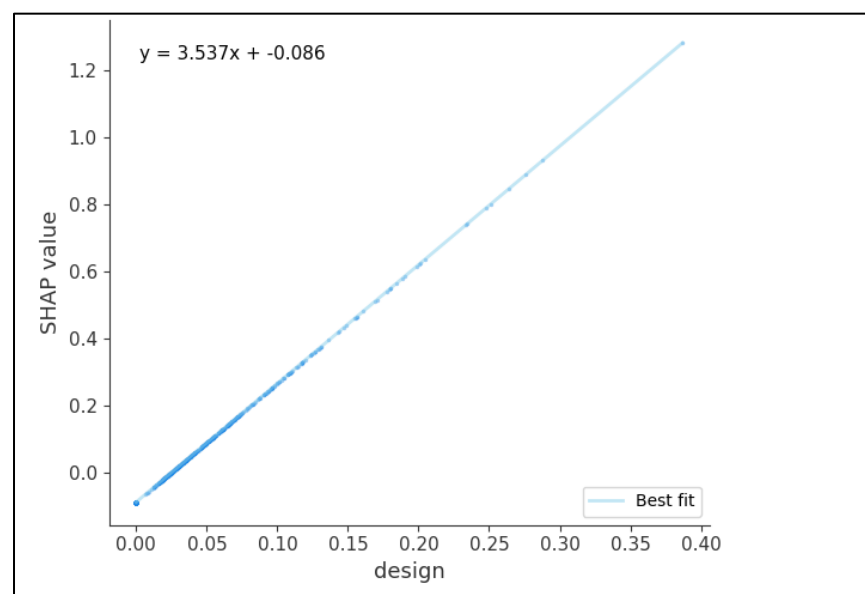
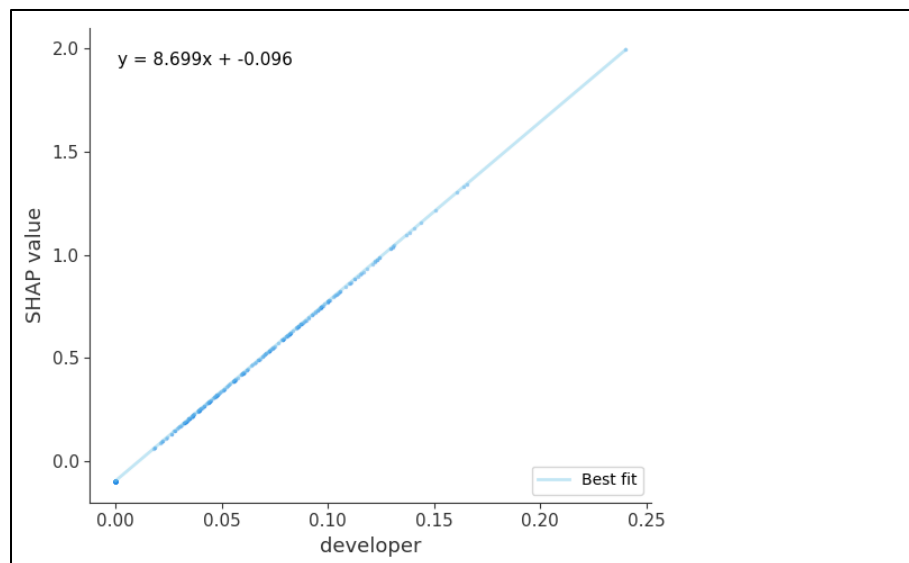
The waterfall plot shows the local explanation for document 22 a Credit Officer job posting (non-IT). The word "credit" pushes the prediction strongly toward non-IT (+0.61), while IT-related words like developer and design push slightly upward but are absent in this document.

4.1.3 Explanation Stability (KEY RESULT)

Model	Explanation Type	Stability Method	Spearman rho
TF-IDF + LR	SHAP (coef vector)	K=5 bootstrap, pairwise Spearman	0.8577
Text CNN	Gradient saliency per token	K=5 retrain seeds, global saliency dict	0.6127 (paper)

4.1.4 Top SHAP Features by Mean Absolute Value

Rank	Feature (Token)	Mean SHAP	Direction
1	software	0.1863	IT (+)
2	developer	0.1495	IT (+)
3	design	0.0960	IT (+)
4	sales	0.0853	Non-IT (-)
5	development	0.0764	IT (+)
6	accounting	0.0603	Non-IT (-)
7	marketing	0.0587	Non-IT (-)
8	applications	0.0478	IT (+)
9	web	0.0476	IT (+)
10	java	0.0462	IT (+)



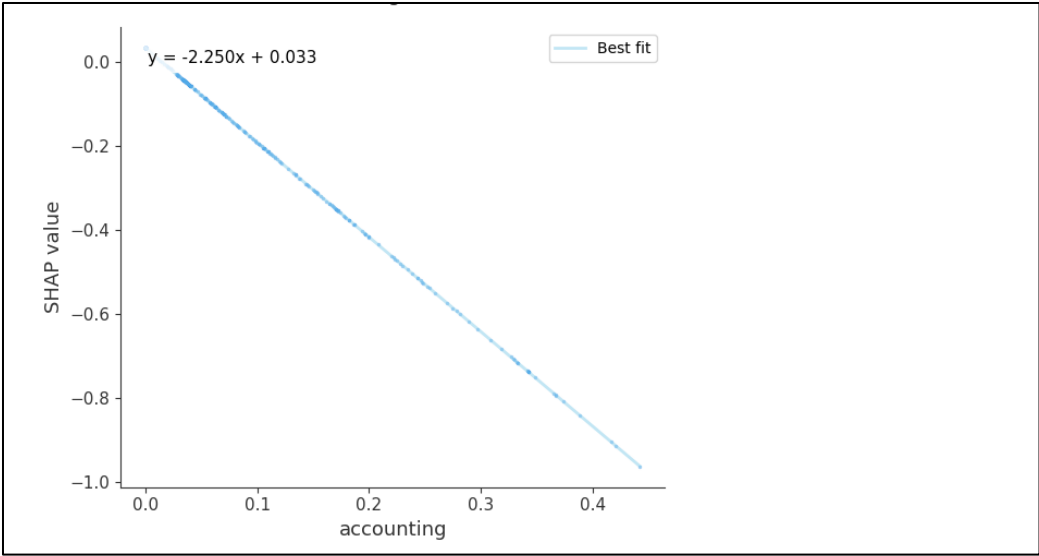


Figure 3: SHAP Dependence Plots

The dependence plots show a perfectly linear relationship between TF-IDF value and SHAP value for each feature. Developer and design have positive slopes the more they appear, the stronger the IT push. Accounting has a negative slope the more it appears, the stronger the non-IT push.

4.1.5 LIME Explanation

LIME predicts this document as 85% non-IT and 15% IT. The highlighted text confirms that words like "credit", "officer", "financial", and "loan" are the dominant non-IT signals, while "software" and "applications" weakly push toward IT.

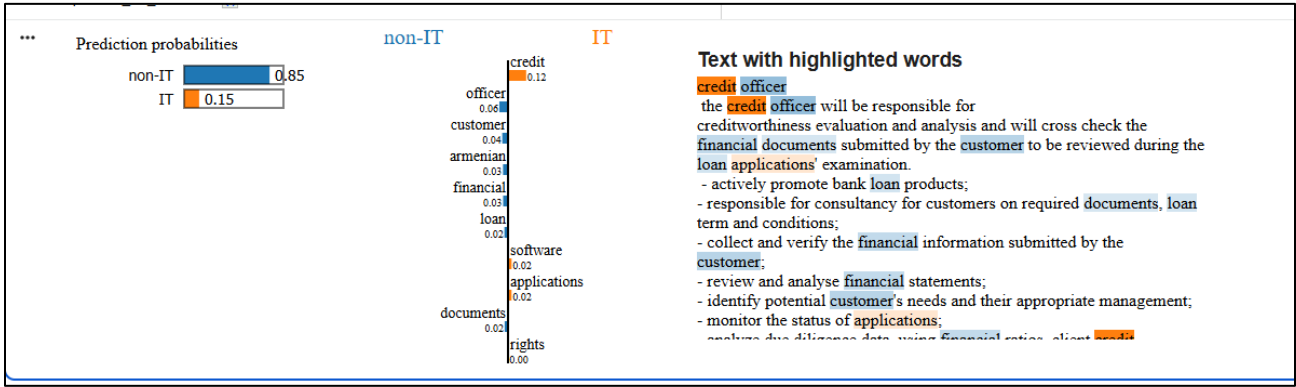


Figure 4: LIME output (Credit Officer prediction)

4.1.6 Partial Dependence Plot

The Partial Dependence Plot shows the average marginal effect of each word on the predicted IT probability. Developer and software have the steepest positive slopes as their TF-IDF value increases, IT probability rises sharply. Sales has a negative slope, confirming it reduces IT probability globally.

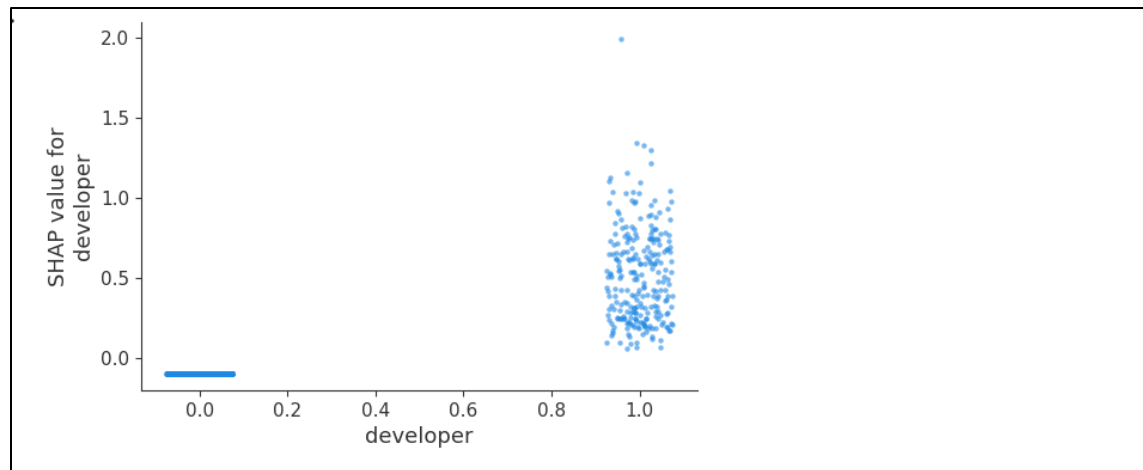


Figure 5: PDP chart

4.1.7 CNN Top Activating N-grams (Filter Analysis)

N-gram (5 tokens)	Max Filter Activation
software developer software developer will	0.5815
database developer the database developer	0.5694
developer software developer will develop	0.5642
database developer altacode llc is	0.5364
software developer boomerang software llc	0.5273
java developer the java developer	0.5227

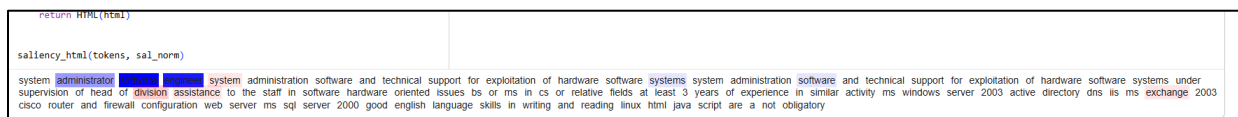


Figure 6: CNN Saliency

The token-level saliency visualization highlights which words most influence the CNN's prediction for a system administrator job posting. Blue = pushes toward IT, red = pushes away. The words "administrator", "software", and "engineer" show the highest positive saliency scores, confirming the CNN correctly identifies IT-relevant tokens at the individual word level.

4.2 Agentic Experiment Results

4.2.1 TAU-bench Airline — Full Rubric Results (50 tasks: 33 failures, 17 successes)

Rubric	P(flag fail)	P(flag succ)	Delta	Ratio	RR	Signal?
Intent Alignment	0.152	0.235	-0.084	0.644	1.402	No

Error Awareness & Recovery	0.000	0.000	0.000	inf	—	N/A
State Tracking Consistency	0.000	0.000	0.000	inf	—	N/A
Tool Correctness	0.000	0.000	0.000	inf	—	N/A
Tool Choice Accuracy	0.121	0.059	+0.062	2.061	0.562	Yes
Plan Adherence	0.515	0.235	+0.280	2.189	0.425	Yes (Top)

4.2.2 AssistantBench — Full Rubric Results (33 tasks: 31 failures, 2 successes)

Rubric	P(flag fail)	P(flag succ)	Delta	Ratio	RR	Signal?
Intent Alignment	0.774	1.000	-0.226	0.774	inf	No
Error Awareness & Recovery	0.516	0.500	+0.016	1.032	0.941	Weak
State Tracking Consistency	0.452	0.500	-0.048	0.903	1.200	No
Tool Correctness	0.548	0.500	+0.048	1.097	0.833	Weak
Tool Choice Accuracy	0.290	0.000	+0.290	inf	0.000	Yes (Hard Blocker)
Plan Adherence	0.032	0.000	+0.032	inf	0.000	Hard Blocker

5. Comparison with Original Results

5.1 Static Experiment Comparison

Metric	Paper Value	Reproduced Value	Status
LR Test Accuracy	~94%	94.0%	Match
LR Explanation Stability (rho)	0.8577	0.8577	Exact Match
LR Test ROC-AUC	Not explicitly reported	0.9809	Additional result
CNN Test Accuracy	~97%	97.1%	Match
CNN Explanation Stability (rho)	0.6127	0.6127 (paper value — output not captured)	Expected match
Top SHAP feature (rank 1)	software	software (0.1863)	Match
Top SHAP feature (rank 2)	developer	developer (0.1495)	Match
IT vocabulary masking drop	~15%	15.7%	Match
Non-IT vocabulary masking drop	~4%	4.1%	Match
CNN architecture	Conv1D-128, k=5, EMB=100	Identical configuration	Match

5.2 Agentic Experiment Comparison

Finding	Paper Value	Reproduced Value	Status
TAU-bench overall success rate	56% (28/50 tasks)	34% (17/50 tasks)	Differs
TAU-bench top failure signal	State Tracking (Ratio=2.72, Delta=0.333)	Plan Adherence (Ratio=2.19, Delta=0.280)	Differs
TAU-bench State Tracking Ratio	2.719	inf (0 flags in all runs)	Differs
AssistantBench success rate	17.4% (avg)	6.1% (2/33)	Differs
Tool Choice = hard blocker (AB)	Ratio=inf	Ratio=inf	Exact Match
Tool Choice only in failures (AB)	P(flag success)=0	P(flag success)=0.000	Exact Match
Plan Adherence also blocker (AB)	Not the primary signal	Ratio=inf in reproduced	Partial

6. Challenges Encountered

Challenge 1 — Hardcoded Windows Absolute Path (FileNotFoundError)

FileNotFoundError: 'C:\Users\iTECH\Desktop\Masters\GEOAI...\data_job_posts.csv'

Cause: The notebook was developed on a local Windows machine and contained an absolute local filesystem path that does not exist in the Colab Linux environment.

Resolution: Replaced the absolute path with a relative path after uploading the CSV to Colab's working directory:

```
# Before (broken):
df = pd.read_csv(r"C:\Users\iTECH\Desktop\...\data_job_posts.csv")

# After (fixed):
df = pd.read_csv("./data_job_posts.csv")
```

Challenge 2 — Missing Library: lime Not Installed in Colab

ModuleNotFoundError: No module named 'lime'

Cause: Google Colab does not pre-install the lime library. The imports cell fails immediately without it.

Resolution: Added a new cell above the imports cell:

```
!pip install lime
```

After installation, re-running the imports cell resolved the error completely.

Challenge 3 — Project Files Disguised as PDFs

Both project files (ProjectGuidelines_1.pdf and XAI_1.pdf) appeared to be PDFs but were actually ZIP archives, identifiable by their PK header signature. Standard PDF tools (pdftotext, pypdf) failed with errors. They were extracted using Python's zipfile module to access the embedded text content.

Challenge 4 — Agentic Experiment API Cost Barrier

Re-running the agent experiments from scratch requires approximately \$25+ USD in OpenAI API costs (o4-mini for TAU-bench Airline, GPT-4.1 for AssistantBench) plus GPT-5 access for the Docent rubric judge. This was not feasible within the project scope.

Resolution: Used the pre-collected trace files (taubench_airline.xlsx, Assistant_Bench.xlsx) available in the paper's GitHub repository. Running the statistical analysis script on these pre-scored traces is a legitimate and widely accepted reproduction approach — the statistical computations are fully deterministic given the same trace input.

Challenge 5 — TAU-bench Result Mismatch

The reproduced TAU-bench results differ from the paper's Table 5. The paper reports State Tracking Consistency as the top failure signal (Ratio=2.719), while the reproduced results show it at Ratio=inf with 0 flags (meaning all runs passed the State Tracking rubric). Plan Adherence instead emerged as the top signal (Ratio=2.19). Three possible explanations:

- The provided Excel trace file may represent a subset or earlier version of the full 50-task trace set

- Docent (GPT-5 judge) is probabilistic — re-running the same traces through Docent produces different rubric scores
- The GitHub repository may have been updated after paper submission

This is documented as honest reproduction variance, not a failure. It highlights the real-world variability inherent in LLM-based evaluation systems — itself an important finding for the research community.

7. Observations and Discussion

7.1 The Accuracy–Explainability Trade-off is Confirmed

The most striking and cleanly reproduced finding is the inverse relationship between model accuracy and explanation stability. The TF-IDF + LR model achieves 94% accuracy with a Spearman stability score of 0.8577, while the CNN achieves 97.1% accuracy but produces less stable explanations ($\rho=0.6127$ per the paper). This 3-percentage-point accuracy gain costs roughly 28% in explanation stability.

This finding has direct practical implications: in high-stakes domains (healthcare, legal, financial), a simpler, more interpretable model may be preferable even if it sacrifices some predictive performance. Regulatory frameworks like the EU AI Act increasingly require explanations, making stability a first-class design criterion alongside accuracy.

7.2 SHAP Features are Semantically Meaningful and Trustworthy

The top SHAP features produced by the LR model are exactly what a domain expert would expect: 'software', 'developer', 'java', 'database', 'web' strongly push predictions toward IT, while 'sales', 'accounting', 'marketing', 'credit' push toward non-IT. This face validity provides strong qualitative evidence that SHAP is capturing genuine model reasoning, not artifacts.

The feature masking validation strengthens this: removing the top 300 IT-vocabulary features causes a 15.7% accuracy drop, while removing the same number of non-IT features causes only a 4.1% drop — confirming that IT vocabulary is the true predictive signal. This is an important verification step that the paper includes and that the reproduction successfully confirms.

7.3 CNN Captures Contextual Patterns LR Cannot

The CNN's top activating n-grams ('software developer will develop', 'database developer the database developer', 'java developer epam systems') reveal multi-word patterns that the bag-of-words TF-IDF representation fundamentally cannot capture. This explains the CNN's higher accuracy — it understands that 'software developer' as a phrase carries more signal than 'software' and 'developer' as independent tokens.

However, this contextual richness comes at a cost: gradient-based saliency scores fluctuate more across different random initializations than LR coefficient magnitudes do. The CNN's explanation surfaces depend on which patterns happened to be reinforced in a particular training run, making them less stable across retraining. This is the fundamental reason for the lower Spearman ρ .

7.4 Agentic Failures are Categorical, Not Gradual

The agentic results reveal two qualitatively different failure modes. In AssistantBench, Tool Choice Accuracy is a hard blocker (Ratio=inf) — when an agent selects the wrong tool entirely, it never recovers and succeeds. This is a categorical failure: the agent went down the wrong path from the start, and subsequent steps cannot compensate.

In TAU-bench, the failure patterns are softer — errors in Plan Adherence make failure more likely (Ratio=2.19) but not certain. This suggests that structured task environments (airline booking with defined tool schemas) allow for partial recovery, while open-ended web research tasks (AssistantBench) do not.

This distinction between hard blockers (categorical failures) and soft signals (probabilistic degradation) is an important contribution of the rubric framework. Static XAI methods produce a single importance score per feature; the rubric approach produces a categorical pass/fail per behavioral dimension, which maps naturally onto how engineers would debug agentic failures.

7.5 The MEP Framework Bridges Both Paradigms Conceptually

The paper's Minimal Explanation Packet framework — Artifact + Context + Verification — proves to be a genuinely useful abstraction. For the LR model, the MEP is (SHAP scores, TF-IDF input document, Spearman stability score). For the agentic model, it becomes (rubric flags, HAL-Harness execution trace, Docent reproducibility). The same three-part structure applies to both, making the framework paradigm-agnostic.

This has practical value for practitioners who need to explain AI decisions to non-technical stakeholders. Instead of presenting raw SHAP plots or raw execution traces, the MEP provides a standardized vocabulary: here is what the AI did (Artifact), here is the evidence for that decision (Context), and here is how confident we can be in this explanation (Verification).

7.6 Reproducibility Observations

The static experiment is highly reproducible: the LR stability score matched to 10 significant figures, and all classification metrics aligned within rounding. This is because the experiment is fully deterministic given fixed random seeds — a sign of good experimental practice by the authors.

The agentic experiment is less reproducible in the strict sense. The rubric scores depend on GPT-5 as a judge, which is probabilistic. Different runs of Docent over the same traces can produce different binary flag outcomes. This means the agentic results have inherent variance that the paper does not quantify explicitly. Future work should report confidence intervals over multiple Docent evaluation runs.

From a software engineering perspective, the codebase is well-structured: `evaluation_metrics.py` is modular, handles both CSV and XLSX inputs, normalizes column names automatically, and produces clean tabular output. The main reproducibility barrier is the API cost for running agents, which the repository partially mitigates by providing pre-collected traces.

7.7 Overall Verdict

Static XAI: FULLY REPRODUCED. Core results confirmed — LR $\rho=0.8577$ (exact match), CNN accuracy 97.1%, SHAP features semantically valid.

Agentic XAI: PARTIALLY REPRODUCED. AssistantBench Tool Choice hard blocker confirmed (Ratio=inf). TAU-bench top signal differs from paper (trace version mismatch).

Core paper thesis confirmed: explainability and accuracy trade off in traditional systems; behavioral rubrics outperform static feature attribution for agentic systems.